Firewatch: detailed task list

Team: Dave (lead, Rust support, data, demo) · Isaac (Rust) · Patrick (frontend) · Follows the PRD timeline, Day 1 (Oct 21) to Day 9 (Oct 29), plus the Oct 31 demo · Status taken from repo `Chibey-max/Firewatch` at commit `79ce029` (Oct 22, 21:37)

Dave Isaac Patrick All DONE in the repo PARTLY started OPEN not started CHECK written but not tested live STRETCH only after the must-haves pass

How to use this list

- Every task has an **ID**, an **owner**, an **estimate**, its **dependencies**, numbered **steps**, the **files** it touches, and a **Done when** line. A task is only done when its Done-when line is true, checked by someone other than the owner.
- Track each task as a GitHub issue titled with its ID (e.g. `D3-1 LND REST connector`). Work on a branch named after the ID (`d3-1-lnd-connector`), open a PR, get one review, and merge only when CI is green.
- Each day ends with a **gate**. If a gate fails, the next morning starts with the failing item, not new work.
- The appendices (A–H) hold the reference material the tasks point to: repo status, advisory data and test cases, the JSON contract, config, the demo network, the demo script, Q&A, and the packing list.

> **Read first: the repo has moved ahead of the PRD in a different direction.** The PRD is LND-first (an LND 0.18.x node turns red). The repo is CLN-first: the engine, a CLN connector and custom Polar images for CLN 26.06.7 and 26.06.9 are built, but there's no LND connector yet. This list **keeps both**:
>
> - **Primary red→green demo: Core Lightning 26.06.7 → 26.06.9.** It's already built, it's the exploited-in-the-wild story, and it matches the Oct 2 hook.
> - **LND path (still a PRD must-have):** an LND 0.18.x node, four Lightning Labs advisories, and the commit-hash check. It supplies the verified amber ("Can't tell") and the honesty moment, which the unverified FW-2026-005 can't.
>
> If the team wants only one path, cut the other by end of Day 3 (Oct 23) and delete its tasks.

> **Pre-building rule.** The original brief says "all code is written on the build day", and the repo already holds about 1,600 lines of Rust. Task D2-1 gets the organizers' answer in writing. If pre-building isn't allowed, stop committing code, follow the contingency in D2-1, and come back to this list.

Day 1: Wed 21 Oct: Setup & validation

Past. Status checked against the repo on 22 Oct.

ID	Owner	PRD task	Status	Evidence / what's left (moved to)
D1-1	Dave	Confirm Polar runs LND 0.18.x; document the exact version	OPEN	No record in the repo. Polar CLN custom images exist (`demo/polar/`), but there's no LND check. → **D2-2**
D1-2	Isaac	Shared repo, Rust structure, CI skeleton	PARTLY	3-crate workspace done; `cargo test` passes 32 tests; clippy and fmt clean. **No CI.** → **D2-5**
D1-3	Patrick	Frontend project; red/amber/green colour system	OPEN	No `web/` folder in the repo. If it exists locally, push it. → **D2-4**
D1-4	Dave	Lock 4–5 primary-source advisories; draft TOMLs	PARTLY	6 TOMLs: 4 verified (LDK ×2, CLN 26.06.8, CLN 26.06.9), 2 unverified (FW-004, FW-005). The Lightning Labs LND advisories are missing. → **D3-5**
D1-5	Isaac	LND GetInfo client skeleton	OPEN	A CLN connector was built instead (`crates/firewatch-connectors/src/cln.rs`). → **D3-1**

Day 2: Thu 22 Oct: Core connectors

PRD open-items deadline · masterclass · finish tonight or first thing Fri

Goal: the rules question answered, the Polar network running with real nodes, CLN checked live (not from a snapshot), CI on, and the JSON contract agreed.

`D2-1` **Dave** **Get the event rules in writing** 0.5 h · no deps · **OPEN** · do this first

1. Message the GDG Kaduna / Btrust organizers on their official channel (WhatsApp group, email or form contact). Send exactly these questions:

   ```
   Hi, we're a Track 1 (Greenfield) team for the Bitcoin Build-a-thon on Oct 29.
   1. Can teams write code before Oct 29, or must all code be written on the day?
   2. If prior code isn't allowed, can we bring research, data files and environment setup?
   3. Is judging on Oct 29 or at the Oct 31 demo showcase, and what are the judging criteria?
   4. What time does the build day start and end, and is there power and Wi-Fi at the venue?
   5. What are the prizes, and is a public GitHub repo required for submission?
   Thank you!
   ```

2. Save the answer (screenshot plus text) in `docs/event-rules.md` with the date and the name of who answered.
3. **If pre-building is allowed:** carry on with this list.
4. **If it isn't:** stop merging code. Tell the organizers what's already in the repo and ask whether to (a) declare it as prior work, (b) start a fresh repo on the day, or (c) move to the open-source contribution track. Follow their answer, and tell the team the same day.

Done when: `docs/event-rules.md` holds a written answer to Q1 and Q3, and the team has read it.

D2-2 Dave **Build the demo Polar network and record every connection detail** 2 h · no deps · CHECK

1. Record the Polar version (Help → About) and the Docker version (`docker version`).
2. Build the CLN images if you haven't yet (commands in the README): `firewatch/clightning:26.06.7` and `:26.06.9`. In Polar, go to **Manage Images** → **Custom Nodes** and add both.
3. Open Polar's LND version dropdown and write down **every 0.18.x, 0.19.x, 0.20.x and 0.21.x version listed**.
4. Create the network `firewatch-demo` with these nodes:
   - 1 bitcoind
   - `old-cln` (custom 26.06.7)
   - `current-cln` (custom 26.06.9)
   - `lnd-old`: **0.18.5**, or the highest 0.18.x Polar has
   - `lnd-new`: the newest LND Polar has. It must be ≥ 0.19.0, and should be ≥ 0.20.2 and not 0.21.0

   Start the network.
5. For each CLN node, create a restricted rune. Use Polar's node terminal, or run `docker exec -it polar-n<N>-old-cln lightning-cli --network=regtest createrune restrictions='[["method=getinfo","method=listconfigs"]]'`. Save each rune to `~/firewatch-secrets/<node>.rune`, which is outside the repo.
6. From each node's **Connect** tab, copy into `docs/demo-env.md`:
   - the REST URL and host port, and the gRPC host port
   - the TLS cert path (LND)
   - the `readonly.macaroon` path (LND)
   - the Docker container name

   Expected LND paths look like `~/.polar/networks/<id>/volumes/lnd/<name>/tls.cert` and `…/data/chain/bitcoin/regtest/readonly.macaroon`. Verify them; don't assume.
7. Create `firewatch.toml` (git-ignored) from `firewatch.example.toml` with the two CLN nodes, then run `cargo run -- check`. Paste the full output into `docs/demo-env.md`.
8. For each LND node, run:

   ```
   curl --cacert <tls.cert> -H "Grpc-Metadata-macaroon: $(xxd -ps -u -c 1000 <readonly.macaroon>)" https://127.0.0.1:<rest-port>/v1/getinfo
   ```

   If curl rejects the cert name, use `https://localhost:<port>`. Save the full JSON response as `crates/firewatch-connectors/tests/fixtures/lnd_getinfo_<version>.json`; Isaac needs it for D3-1.
9. Note the `version` and `commit_hash` fields from each response. Then check whether the official image reports the tag's commit:

   ```
   git ls-remote https://github.com/lightningnetwork/lnd 'refs/tags/v0.18.5-beta^{}'
   ```

   Compare the hash it prints with `commit_hash` and write **MATCH** or **NO MATCH** in `docs/demo-env.md`. This decides the build-check design in D4-1.

Files: `docs/demo-env.md` (new), `crates/firewatch-connectors/tests/fixtures/*.json` (new). Never commit runes, macaroons or certs.

Done when: `cargo run -- check` shows `old-cln` as AFFECTED and `current-cln` as NOT AFFECTED from **live** nodes; both LND GetInfo JSON files are saved; and the MATCH / NO MATCH result is written down.

D2-3 Dave **Prove the on-stage "upgrade" works (time-box 1.5 h)**

1.5 h · needs D2-2 · OPEN

1. Try each method in order and stop at the first one that keeps the node's data and name:
   - **A.** Polar: stop the node and change its image or version in the UI, if Polar offers this.
   - **B.** Stop the network. In `~/.polar/networks/<id>/docker-compose.yml`, change `old-cln`'s image to `firewatch/clightning:26.06.9`. Start the network again and check that Polar didn't overwrite the file.
   - **C.** Leave Polar for this node: a small `demo/compose/docker-compose.yml` with bitcoind and `old-cln`, where the upgrade is `docker compose up -d` after changing the tag.
2. Time the upgrade from command to `getinfo` answering again. It must be under 40 s.
3. Write the exact working steps in `docs/demo-runbook.md` under "Upgrade".

Done when: `old-cln` reports `v26.06.9` after the upgrade, Firewatch shows it NOT AFFECTED without any config change, and the steps are written down.

D2-4 Patrick **Create `web/` and adopt the real JSON contract**

2.5 h · no deps · OPEN

1. Run `npm create vite@latest web -- --template react-ts` from the repo root. Add Tailwind (your stack), plus eslint and prettier with defaults.
2. In `web/vite.config.ts`, add the proxy `server.proxy = { '/api': 'http://127.0.0.1:7070' }`. This means no CORS is needed later.
3. The contract is **what the Rust code already outputs**, not the JSON sketch in the PRD. Generate the first fixture with:

   ```
   cargo run -- check --nodes examples/nodes.json --json > web/src/fixtures/report.mixed.json
   ```

4. Copy the TypeScript types from **Appendix C** into `web/src/types/report.ts`. Leave the fields marked "planned" as optional (`?`).
5. Hand-write three more fixtures in `web/src/fixtures/`:
   - `report.green.json`: every node not affected
   - `report.error.json`: one node with `error` set, `version: null` and `status: "cant_tell"`
   - `report.empty.json`: `nodes: []`
6. Set the colour tokens in `web/src/index.css`:
   - Affected: red `#B42318`, icon ✕
   - Can't tell: amber `#B54708`, icon !
   - Not affected: green `#067647`, icon ✓

   The verdict words are exactly **Affected**, **Can't tell** and **Not affected**. Never show colour alone: always icon + word.
7. Push on branch `d2-4-web-scaffold` and open a PR.

Done when: `cd web && npm run build` passes; the four fixtures and `report.ts` exist; and Isaac has approved the PR (that approval is the "JSON contract agreed" PRD item).

D2-5 Isaac **CI and secret-safe .gitignore**

0.75 h · no deps · OPEN

1. Add `.github/workflows/ci.yml` that runs on push and PR. It needs two jobs:
   - `rust`: `cargo fmt --check`, `cargo clippy --workspace -- -D warnings`, `cargo test --workspace`.
   - `web` (only if `web/` exists): `npm ci && npm run build` in `web/`.
2. Add to `.gitignore`: `*.macaroon`, `*.rune`, `tls.cert`, `*.nsec`, `.env`, `web/node_modules` and `web/dist`.
3. In GitHub settings, protect `main`: require a PR, one approval and passing CI. Add Isaac and Patrick as collaborators if they aren't already.

Done when: the CI badge is green on `main`, and a direct push to `main` is rejected.

D2-6 All **Masterclass (Oct 22)**

as scheduled

Ask one mentor: *"If a tool told you your node was affected by an advisory, what would make you trust it, and what would make you ignore it?"* Write the answer in `docs/mentor-notes.md`.

Gate: end of Day 2 (PRD §9 open items)
☐ Event rules answered in writing (D2-1) ☐ Polar network up, with LND 0.18.x confirmed (D2-2) ☐ Repo shared, CI green (D2-5)
☐ First TOML files reviewed by Dave and Isaac (the 4 verified ones) ☐ JSON contract agreed (D2-4 approved)

Day 3: Fri 23 Oct: Matching engine → LND path

Isaac ~6 h · Dave ~5 h · Patrick ~4 h

Goal: the PRD's Day-3 engine tasks are already done (loader, ranges, three verdicts, CLI). Use the day for what's missing: the LND connector and the Lightning Labs advisories. Decide today whether to keep both paths (see the note on page 1).

D3-0 All 15-min check-in: keep both demo paths?

0.25 h · OPEN

Decide: (a) CLN red→green plus LND amber/honesty (recommended), (b) CLN only, or (c) LND only. Record the decision at the top of `docs/demo-runbook.md`. If (b), drop D3-1, D3-5, D4-1, D4-6. If (c), drop D2-3 and the CLN beats in Appendix F.

D3-1 Isaac LND REST connector (read-only macaroon)

3.5 h · needs the D2-2 fixture JSON · OPEN

1. Create `crates/firewatch-connectors/src/lnd.rs` and add `pub mod lnd;` to `lib.rs`.
2. Add `LndConfig { name, url, tls_cert: PathBuf, macaroon: PathBuf }` with `#[serde(deny_unknown_fields)]`. Add the `NodeConfig::Lnd(LndConfig)` variant, and update `name()`, `implementation()` and `fetch()`.
3. **Refuse admin credentials:** if the macaroon file name is `admin.macaroon`, return `ConnectorError::NotReadOnly("use readonly.macaroon")`. (Stretch: decode the macaroon ID's permission list instead of relying on the file name.)
4. Build a **per-node** client, because each LND node has its own self-signed cert:

```
reqwest::Client::builder()
    .add_root_certificate(reqwest::Certificate::from_pem(&std::fs::read(&cfg.tls_cert)?)?)
    .timeout(Duration::from_secs(10))
    .build()?
```

   `fetch()` currently passes one shared client. For LND, ignore it and build this one in `LndClient::new`.
5. Call `GET {url}/v1/getinfo` with the header `Grpc-Metadata-macaroon: <hex of the macaroon file bytes>`. Use the `hex` crate or a two-line encoder.
6. Map the response into `NodeInfo`:
   - `version` ← `version`, kept as-is, e.g. `0.18.5-beta commit=v0.18.5-beta`
   - `network` ← `chains[0].network`
   - `features` ← `Some(set of the features object's keys parsed as u32)`
   - `config` ← `None` (lnd.conf isn't readable over RPC)
   - **new** `commit_hash` ← `commit_hash`
7. In `crates/firewatch-core/src/node.rs`, add `#[serde(default)] pub commit_hash: Option<String>`. Fix every `NodeInfo { … }` literal (engine tests, `tests/advisories.rs`, CLN connector) by adding `commit_hash: None`. `examples/nodes.json` keeps working because of the serde default.
8. Map errors: connection refused or timeout → `Http`; HTTP 401/403 or a body containing "verification failed" → `Rpc` with the message; missing `version` → `Unexpected`.
9. Add tests: parse `tests/fixtures/lnd_getinfo_*.json` into `NodeInfo` (version, network, features non-empty, commit_hash 40 hex chars), and check that `admin.macaroon` is refused.
10. Add an LND block to `firewatch.example.toml` (see Appendix D) and a README line.

Done when: `cargo run -- check` against Polar lists `lnd-old` and `lnd-new` with their real versions; the tests pass in CI; and with `admin.macaroon` configured, the node is refused with a clear message.

D3-2 Isaac **Advisory format additions for Lightning Labs entries** 1 h · no deps · OPEN

1. Add an optional field `upstream_severity: Option<String>` (e.g. `"T1"`) to `Advisory`, and copy it into `Finding` in `report.rs`.
2. In `tests/advisories.rs`, `verified_advisories_quote_real_sources` only allows `https://github.com/`, so it will fail on Lightning Labs entries. Replace it with an allowlist:
   - `https://github.com/ElementsProject/lightning/`
   - `https://github.com/lightningdevkit/rust-lightning/`
   - `https://github.com/lightningnetwork/lnd/`
   - `https://security.lightning.engineering/`
3. Add a test that checks every verified advisory's `source` uses one of these prefixes.

Done when: a TOML containing `upstream_severity = "T1"` and a `security.lightning.engineering` source loads, and its verified-source test passes.

D3-5 Dave **Add four Lightning Labs T1 advisories (FW-2026-007 to 010)** 3 h · needs D3-2 · OPEN

1. Open `https://security.lightning.engineering/`. For each advisory in **Appendix B, table B2**, open its own page and copy the **title, tier, disclosure date, affected versions, fixed versions and any remediation sentence word for word**.
2. Save the raw copy as `docs/sources/FW-2026-0xx.md` with the URL, today's date and the copied text. This is your evidence in Q&A.
3. Write `advisories/FW-2026-007.toml` to `010.toml` using the ranges in B2. Rules:
   - `remediation.steps` is a verbatim sentence from the advisory page. If the page has no remediation sentence, quote its "fixed in" line exactly. Never paraphrase.
   - `source` is the advisory's own page URL, not the index.
   - `severity`: pick it from the tier definition on the Lightning Labs page, and write one line of reasoning in the sources file.
   - FW-2026-010 (interceptor) gets a `manual` condition. The `check_hint` is our instruction, so say so: *"Check whether any app on this node uses LND's HTLC interceptor (e.g. requireinterceptor in lnd.conf or an app that intercepts HTLCs)."*
4. Set `verified = true` only after Isaac has re-checked each file against its sources file.
5. Add one test per advisory to `crates/firewatch-core/tests/advisories.rs` using the test cases in table B2, and add the IDs to `TESTED`.

Done when: `cargo run -- validate` lists 007–010 as verified; `cargo test` passes; and `cargo run -- check` on `lnd-old` (0.18.5) shows FW-2026-009 **Affected** and FW-2026-010 **Can't tell**.

D3-6 Dave **Resolve the two unverified advisories and the CLN ranges** 1.5 h · no deps · PARTLY

1. **FW-2026-004 (CLN "go offline").** Look for the official statement (CLN GitHub, the Blockstream blog, or the Discord message confirmed by Mark Erhardt as reported). If you find it:
   - quote it
   - use the `--offline` wording, not "take offline"
   - fix the range: the title says 26.04 but `affected` is `< 26.06`
   - set `verified = true`

   If you can't find it by end of Day 3, delete the file and its test.
2. **FW-2026-005 (LND cooperative close + reorg).** Look for the primary disclosure: Bastien Teinturier's post on Delving Bitcoin or the lightning-dev list, and the lnd release notes for 0.21.0. If found, quote it and verify. If not, keep it `verified = false`. It must not be in the demo.
3. **CLN ranges.** FW-003 and FW-006 use "everything below the fix". Read the 26.06.7, 26.06.8 and 26.06.9 release notes once more. If any of them names an earliest affected version, add a lower bound (`>= X, < Y`). Write your conclusion as a comment at the top of each TOML.

Done when: each of the three items has a written conclusion in the TOML header comment, and `validate` shows no advisory that is both UNVERIFIED and in the demo.

D3-7 Patrick Static dashboard components on fixtures

4 h · needs D2-4 · OPEN

1. Components go in `web/src/components/`:
   - `AppHeader`: product name, "Checked N s ago" from `checked_at`, and the count of advisories that apply.
   - `NodeCard`: header coloured by `node.status`, showing icon + word + node name + implementation + version. Implementation labels: `cln` → Core Lightning, `lnd` → LND, `ldk` → LDK.
   - `VerdictBadge`: {status} → colour, icon and word.
   - `FindingRow`: advisory ID, title, a severity chip (plus `upstream_severity` when present), an **"Exploited in the wild"** red chip when `exploited_in_wild`, a grey **"Unverified"** chip when `verified` is false, and the `reason` sentence.
   - `RemediationQuote`: only for `affected`. The label reads "Developers say:", then a blockquote of `steps`, then a "Source" link.
   - `ManualChecks`: only for `cant_tell`. The label reads "Check by hand:", then a bulleted `manual_checks` list, then a "Source" link.
   - `NotAffectedGroup`: collapses all `not_affected` findings under "N advisories don't apply ▸". Expanding it shows each one's `reason`.
   - `NodeError`: when `error` is set, shows "Couldn't read this node", the error text, and an amber card. It is **never green**.
2. Order the findings as the server already sends them (worst first). Don't re-sort.
3. Add a fixture switcher: `?fixture=mixed|green|error|empty` in the URL loads the matching JSON file. This is your demo fallback if the server dies.
4. Projector rules: body text ≥ 18 px; node card header ≥ 24 px; check at 1280×720 with no horizontal scroll.

Done when: all four fixtures render correctly at 1280×720, and someone who hasn't seen the code can say in under 10 seconds which node is in trouble and what to do.

Gate: end of Day 3

☐ D3-0 decision recorded ☐ LND nodes read live (D3-1) ☐ 007–010 verified, with tests (D3-5) ☐ 004 and 005 resolved (D3-6) ☐ Components render all fixtures (D3-7)

Day 4: Sat 24 Oct: Honesty & integration

Masterclass · Isaac ~6 h · Dave ~5 h · Patrick ~3 h

Goal: the commit-hash check works, there's a local API, and the dashboard shows **live** verdicts from Polar.

D4-1 Isaac **Release table and build check (commit hash mismatch → Can't tell)**

3 h · needs D3-1, D2-2 MATCH result · OPEN

1. Add `scripts/lnd-releases.sh`. It runs `git ls-remote --tags https://github.com/lightningnetwork/lnd 'refs/tags/v0.1[6-9]*' 'refs/tags/v0.2*'`, keeps only the lines ending in `^{}` (the commit an annotated tag points to), and writes `releases/lnd.toml`:

   ```
   [[release]]
   version = "v0.18.5-beta"
   commit  = "<40-hex>"
   ```

2. Add `crates/firewatch-core/src/releases.rs`: `ReleaseTable::load(path)` and `expected_commit(&Version) -> Option<&str>`. Match on the parsed version, so `0.18.5-beta` finds `v0.18.5-beta`.
3. Add `enum BuildCheck { Match, Mismatch { expected, actual }, UnknownRelease, NotChecked }`, serialized in snake_case, plus `NodeReport.build_check: BuildCheck`.
   - `NotChecked`: the node has no `commit_hash` (CLN, snapshots).
   - `UnknownRelease`: the version isn't in the table, e.g. a release newer than the table. Verdicts stay normal, and the UI shows a note.
   - `Mismatch`: **every** finding for this node becomes `CantTell`, with reason *"This node reports version X, but its build (commit abc1234) is not the official X release (def5678). Firewatch can't trust the version number."* and manual check *"Confirm where this binary came from, or reinstall from the official release."*
4. **If D2-2 was NO MATCH** (Polar's images don't report the tag commit): don't ship the hash comparison, because it would flag every node. Fall back to checking the `commit=` part of the version string. If it isn't exactly `v<version>` (e.g. `v0.21.1-beta-1-gabc123`), return `Mismatch`. Write this limitation in the README.
5. Add a `--releases` flag to the CLI (default `releases/lnd.toml`). `check` and the report builder take the table, and the engine stays pure.
6. Add tests: match → normal verdicts; mismatch → all CantTell with that reason; unknown release → normal verdicts plus `UnknownRelease`; no hash → `NotChecked`.
7. STRETCH CLN: a version with a git-describe suffix (`v26.06.7-12-gabc123`) or `-modded` → `Mismatch` ("not an official release build").

Done when: on Polar, `lnd-old` and `lnd-new` show `build_check: "match"`; the four tests pass; and the JSON contains `build_check` (tell Patrick).

D4-2 Isaac **`firewatch serve`: local JSON API with re-check loop**

3 h · no deps · OPEN

1. Dependencies:
   - `axum = "0.8"`
   - `tower-http = { version = "0.6", features = ["fs"] }`
   - tokio features plus `"net", "time", "sync"`
2. Move `check_live` out of `main.rs` into a shared function that `check` and `serve` both call.
3. New subcommand: `serve --config firewatch.toml --listen 127.0.0.1:7070 --interval 15 [--web web/dist] [--include-unverified]`. Refuse a `--listen` address that isn't loopback unless `--allow-remote` is passed. This is our own BTCPay lesson.
4. Keep the state as `Arc<RwLock<Option<Report>>>`. A background task rebuilds the report every `--interval` seconds.
5. Routes:
   - `GET /api/report` → 200 with the Report JSON, or 503 `{"error":"first check still running"}`.
   - `POST /api/recheck` → runs a check now and returns the new Report. This is the demo's "Re-check now" button. It's read-only toward the nodes.
   - `GET /api/health` → `{"ok":true}`
   - With `--web`: serve `web/dist` at `/` through `ServeDir`, so the demo is one process.
6. Log one line per check: time, worst status, and the number of nodes.

Done when: `curl -s localhost:7070/api/report | jq .nodes[].status` returns the live statuses; `curl -X POST localhost:7070/api/recheck` updates `checked_at`; and stopping a Polar node turns its card into an error within one interval, without crashing the server.

D4-3 Patrick **Wire the dashboard to the live API** — 3 h · needs D4-2 (use fixtures until then) · OPEN

1. Write a `useReport()` hook that fetches `/api/report` every 3 s, keeps the last good report on error, and exposes `{report, loading, error, recheck}`.
2. Handle a 503 by showing a "First check running…" skeleton.
3. Add a **Re-check now** button that POSTs `/api/recheck`, shows a spinner while it runs, and is disabled while busy.
4. Show `build_check`: `mismatch` → an amber banner on the node card with the expected and actual short hashes (7 characters); `unknown_release` → a small grey note, "Release not in Firewatch's table; build not verified".
5. If the API is unreachable, show the banner "Firewatch server not reachable; showing last report from HH:MM:SS". Never clear the screen.
6. Keep `?fixture=` working; it overrides the live API.

Done when: with `firewatch serve` running against Polar, the dashboard shows the live statuses, and after the D2-3 upgrade `old-cln` turns green within 5 s of pressing Re-check now.

D4-4 Dave **Full-path test against the expected-verdict table** — 1.5 h · needs D4-2, D4-3 · OPEN

1. Start Polar, run `firewatch serve`, and open the dashboard.
2. Compare each node with **Appendix E**, line by line. Mark each line ✓ or ✗ in `docs/demo-runbook.md`.
3. Click every Source link and confirm it opens the right page.
4. File every ✗ as a GitHub issue with the label `demo-blocker`, assigned to its owner.

Done when: every Appendix E line is ✓, or has an open demo-blocker issue.

D4-5 Dave **Quote audit: every remediation is word for word** — 1 h · needs D3-5 · OPEN

1. For each verified TOML, open its `source` page, press Ctrl+F, and paste `remediation.steps`. It must be found **exactly**, with the same punctuation and backticks.
2. If it isn't found, fix the TOML to match the page exactly. Never edit the page text to read better.
3. Record "audited <date> by Dave" at the bottom of each `docs/sources/*.md`.

Done when: every verified advisory has an audit line, and Isaac has spot-checked two of them.

D4-6 Dave **A mismatched LND build for the honesty moment (time-box 3 h)** — 3 h · needs D4-1 · OPEN

1. Run `git clone https://github.com/lightningnetwork/lnd` and `git checkout v<lnd-new version>`.
2. Run `git commit --allow-empty -m "firewatch demo: not an official release"`. This changes the commit without changing any code.
3. Build with the Go version named in that release's notes: `make release-install`, or `make install` as a fallback. Then run `lnd --version`. The commit should show something like `v0.21.x-beta-1-g<hash>`.
4. Make a Polar-compatible image `FROM` the same Polar LND image, replacing the `lnd` binary. Find its path first with `docker run --rm --entrypoint sh polarlightning/lnd:<v> -c 'which lnd'`. Tag it `firewatch/lnd:mismatch`.
5. Add it in Polar as a custom node `lnd-unofficial`, start it, and add it to `firewatch.toml`.
6. **Fallback** if this runs past 3 h: save a real GetInfo JSON from `lnd-new`, change only the `commit_hash`, and show it with `check --nodes`, saying out loud that it's a recorded snapshot. Never present a snapshot as a live node.

Done when: `lnd-unofficial` shows every finding as **Can't tell** with the build reason, or the fallback is ready and labelled as a snapshot in the runbook.

D4-7 All **Masterclass (Oct 24)** — as scheduled

Show a mentor the dashboard on fixtures for 2 minutes. Write their first reaction in `docs/mentor-notes.md`.

Gate: end of Day 4

☐ Build check works on Polar (D4-1) ☐ `serve` plus the live dashboard (D4-2, D4-3) ☐ Appendix E all ✓ or ticketed (D4-4) ☐ Quotes audited (D4-5)

Day 5: Sun 25 Oct: Demo path & polish

Must-have freeze at end of day · Isaac ~4 h · Dave ~4 h · Patrick ~4 h

Goal: every must-have works end to end, and the demo can be started by someone who didn't build it.

D5-1 Dave One-command demo start and a click-by-click runbook

2 h · needs D4-2 · OPEN

1. Write `scripts/demo-up.sh`. It should:
   - check that Docker is running
   - check each node in `firewatch.toml` answers (curl getinfo)
   - run `cargo build --release`, then `cd web && npm run build`
   - run `target/release/firewatch serve --web web/dist`
   - open `http://127.0.0.1:7070`

   Polar itself is a GUI, so start the network by hand first; the script says so if a node is down.
2. Write `docs/demo-runbook.md` in this order:
   1. Start Polar → network `firewatch-demo` → Start.
   2. Wait for all nodes to go green in Polar.
   3. Run `./scripts/demo-up.sh`.
   4. Follow the screen-by-screen script (Appendix F).
   5. Upgrade steps (from D2-3).
   6. Reset steps, to take `old-cln` back to 26.06.7 for the next run.
3. Have Patrick run the whole runbook alone on his laptop, while Dave watches without helping.

Done when: Patrick gets from a cold start to the dashboard by following only the runbook, and the reset brings `old-cln` back to red.

D5-2 Isaac Error-handling matrix

2 h · needs D3-1 · PARTLY

Cause each failure on purpose and check what the operator sees. Every case should leave the node's status at `cant_tell` with a plain-English `error`, and the other nodes unaffected:

Cause it by	Expected `error` text starts with
Stopping the node in Polar	"Can't connect to <url>; is the node running?"
Pointing at the wrong port	Same as above
Using the wrong `tls.cert`	"TLS certificate not trusted; check tls_cert for <name>"
Using another node's macaroon or rune	"Credential rejected by the node"
Using an unrestricted rune	Existing `NotReadOnly` message
Using `admin.macaroon`	"use readonly.macaroon"
Blocking with `iptables`/firewall (timeout)	"Timed out after 10 s"

Translate the raw `reqwest` errors in one `impl Display` so the dashboard never shows a Rust debug string.

Done when: all 7 rows behave as listed, with a screenshot of each in the PR.

D5-3 Patrick Dashboard polish for the projector

3 h · needs D4-3 · OPEN

1. Source links open in a new tab with `rel="noopener noreferrer"`, and the link text shows the domain (e.g. "github.com/ElementsProject …").
2. Remediation quotes: serif or italic, a left border, at most 70 characters per line, and inline code rendered from backticks.
3. The Can't-tell card says, in one line, *why Firewatch won't guess*, then shows the hand checks.
4. Test on the actual demo laptop at 1280×720 and 1920×1080, in light room conditions (projectors wash out amber, so check the contrast).
5. Make one screenshot per node state for the slides (D7-2).

Done when: Dave approves it on the demo laptop screen from 3 m away.

D5-4 Dave **Backup video, first cut** — 1 h · needs D5-1 · OPEN

1. Use OBS at 1920×1080, 30 fps, recording the screen plus the mic. Record the full Appendix F script, under 3 minutes.
2. Save it as `firewatch-demo-v1.mp4`. It isn't final; D8-5 re-records it.

Done when: the video plays end to end, and the red, amber and green moments are all visible.

D5-5 Isaac **README: LND config, `serve`, build check** — 0.75 h · needs D4-1, D4-2 · PARTLY

Add the LND config block, the `serve` usage, the `build_check` meanings, and the updated limitations (build check is LND-only; lnd.conf options are unreadable). Update the "Layout" table with `web/`, `releases/` and `scripts/`.

Done when: a new reader can configure an LND node from the README alone.

Gate: end of Day 5. Must-have freeze (PRD: "must-haves only until Day 5")

☐ LND GetInfo with version + commit_hash ☐ 4–5+ verified primary-source advisories ☐ Three verdicts ☐ Commit mismatch → Can't tell
☐ Source link + quoted remediation on every red card ☐ Polar red → upgrade → green ☐ Dashboard readable in under 60 s
If any box is empty, Day 6 stretch work is cancelled and the day goes to that box.

Day 6: Mon 26 Oct: Stretch & hardening

Stretch only if the Day 5 gate passed

D6-1 Isaac **Exposure check: API reachable from outside (BTCPay lesson)** — 2.5 h · STRETCH

1. **CLN:** `parse_configs` keeps only booleans. Add a second map for string values, and read `clnrest-host`. If it's `0.0.0.0` or `::`, report an "exposure" finding as Can't tell. The text: "clnrest is bound to all interfaces. Check that port <p> is not reachable from the internet." Cite the BTCPay 2.4.2 advisory as the reason, in a code comment and in the UI.
2. **LND:** listen addresses aren't readable over RPC. Add an optional `lnd_conf` path to `LndConfig`. If it's set, parse the `restlisten=` / `rpclisten=` lines; if it isn't, return Can't tell with a manual check.
3. **Demo caution:** Polar containers bind to 0.0.0.0 inside Docker, so every Polar node will trip this check. Don't feature it on stage; mention it in Q&A only.

Done when: the tests cover `127.0.0.1` (no finding), `0.0.0.0` (Can't tell) and a missing option (Can't tell).

D6-2 Isaac **Encrypted Nostr DM alert on status change** — 3 h · STRETCH

1. Add the `nostr-sdk` crate, using the version current on crates.io that builds cleanly. Config:

```
[alerts.nostr]
secret_key_file = "/home/you/firewatch-secrets/firewatch.nsec"
recipient = "npub1…"
relays = ["wss://relay.damus.io", "wss://nos.lol"]
```

2. In `serve`, send a NIP-17 private DM only when a node's status gets **worse** (not_affected → cant_tell → affected), and once when it becomes not_affected after being affected. Keep the last status per node in memory to avoid repeats.
3. The message:

```
Firewatch: <node> is AFFECTED by <id> (<title>).
<reason>
Developers say: "<steps>"
<source>
```

4. Test on a phone with a client that supports NIP-17 (Amethyst or 0xchat). Confirm the client on Day 6 before relying on it.
5. The venue needs internet for this. If it's down, skip the phone beat; the dashboard is the demo.

Done when: the phone buzzes within 20 s of `old-cln` being checked as affected, and doesn't buzz again on the next check.

D6-3 Patrick **States and accessibility** 3 h · OPEN

- **Loading:** skeleton cards.
- **Empty:** "No nodes configured. Add one to firewatch.toml."
- **Server down:** the D4-3 banner.
- **Can't tell explainer:** a small "Why amber?" popover: *"Firewatch never shows green when it isn't sure. Something this advisory depends on can't be read from the node, so check it by hand."*
- **Accessibility:** every status has icon + word; focus outlines are visible; contrast ≥ 4.5:1 on the text over every status colour.

Done when: each state can be reached with a fixture or by stopping the server, and all of them look intentional.

D6-4 Dave **Upstream proposal draft for lightning-vulns** 1 h · OPEN

Write `docs/upstream-issue.md` as an issue for `ekzyis/lightning-vulns`, titled *"Proposal: a machine-readable advisory file alongside the README"*. Include:

- why: tools can check nodes automatically
- one example entry in our TOML
- a field-by-field mapping to the README's existing data
- an offer to send the PR

Don't file it until the team agrees. Filing after the demo is fine; on stage you say "proposed upstream".

Done when: Isaac and Patrick have read it and agreed when to file it.

D6-5 All **Dry run #1, timed** 1 h · OPEN

Run Appendix F from a cold start with a stopwatch. Roles: Dave speaks, Isaac drives the terminal and Polar, Patrick drives the dashboard and the phone. Log every stumble as a `demo-blocker` issue. Target ≤ 3:00.

Done when: one complete run is recorded with its time, and the issues are filed.

Day 7: Tue 27 Oct: Pitch & final integration

Feature freeze 18:00 WAT

D7-1 Dave **Final pitch line and 3-minute script** 1.5 h · OPEN

Start from Appendix F. Make every claim match what's on screen, and cut any line that names an advisory not in the demo. The one-liner: *"Fixes ship in days. Operators hear late. Firewatch checks your running node and quotes the official fix."*

Done when: the script is printed, has timings per beat, and reads in ≤ 2:50 aloud.

D7-2 Patrick **Pitch slides (6 slides)** 3 h · needs D5-3 screenshots · OPEN

1. **Hook:** "Oct 2: attackers target unpatched Core Lightning nodes. Aug: funds stolen from LND nodes behind BTCPay."
2. **Problem:** the four gaps (hear late, am-I-affected, the version can lie, the right action changes).
3. **Firewatch:** the three verdicts as three coloured cards, and "never green when unsure".
4. **Live demo:** a title-only slide, so you can switch to the dashboard.
5. **Why trust it:** read-only credentials, official sources only, quoted fixes, the build check.
6. **Next and team:** the upstream proposal, the repo URL and a QR code, then Dave, Isaac and Patrick.

Export to PDF too (`docs/firewatch-pitch.pdf`).

Done when: the deck runs in under 60 s without the demo, and the PDF is in the repo.

D7-3 Isaac **Feature freeze and release candidate** 1 h · OPEN

At 18:00 WAT, tag `v0.1.0-rc1`. After that, only PRs labelled `demo-blocker` merge. Run `cargo test --workspace` and record the exact pass count for the README.

Done when: the tag exists, CI is green on it, and the test count is in the README.

D7-4 All **Three timed rehearsals** 1.5 h · OPEN

Run 1 is normal. Run 2 cuts the internet (no Nostr). Run 3 kills the server mid-demo and switches to `?fixture=`, then to the backup video. Write down the time and the recovery step for each.

Done when: all three runs finish under 3:00, and each failure has a practised recovery.

D7-5 Dave **Judge Q&A sheet** 1 h · OPEN

Use Appendix G. Assign each question an owner, drill them once, and print a copy for each person.

Done when: each person answers their questions in under 20 s without notes.

Day 8: Wed 28 Oct: Buffer & backup

No new features

D8-1 Dave **Final test on the demo laptop, offline** 1.5 h · OPEN

1. Pre-pull every image: `docker images | grep -E 'firewatch|polarlightning'` should list them all. Build the release binary and `web/dist`.
2. Turn off Wi-Fi. Run the full runbook. Everything except Nostr must work.
3. Confirm Appendix E is all ✓ for the last time.

Done when: an offline run succeeds, and `old-cln` has been reset to 26.06.7.

D8-2 Isaac **Repo ready to share** 1.5 h · OPEN

- The README has an exact quick start, the limitations and the test count.
- Remove `docs/googledocs.txt` (it's empty) or fill it.
- Replace the old brief PDF in `docs/` with the revised one.
- The repo is public, and `v0.1.0` is tagged.
- `git log -p | grep -iE 'rune|macaroon|nsec'` shows no secret values.

Done when: a fresh clone, run through the README's quick start, passes `cargo test` and `validate`.

D8-3 Patrick **Final UI and deck** 2 h · OPEN

Fix the remaining UI `demo-blocker` issues only. Put the final deck PDF on the demo laptop, a USB stick, and Google Drive.

Done when: the deck opens from the USB stick on a second laptop.

D8-4 Dave **Registration text and packing** 1 h · OPEN

Update the registration answers if the organizers still need them (from the revised brief). Pack using Appendix H.

Done when: every Appendix H item is ticked and in one bag.

D8-5 All **Final backup video in three places** 1 h · OPEN

Re-record with the final build. Store it on the demo laptop desktop, a USB stick, and Google Drive (link-viewable). Test playback from each.

Done when: all three copies play.

Day 9: Thu 29 Oct: Build day, Stonetech Square, Kaduna

Adjust times once the organizers publish them

When	Who	What
Start − 60 min	Dave	Arrive. Find power. Test the venue Wi-Fi and your hotspot. Test the projector with HDMI and USB-C.
Start − 45 min	Isaac	Start Polar → run `demo-up.sh` → Appendix E check. Reset `old-cln` to 26.06.7.
Start − 30 min	Patrick	Dashboard on the projector at its real resolution. Fix zoom and font size. Deck loaded.

Build hours	All	Follow the organizers' format. If pre-building was allowed: only low-risk fixes from the `demo-blocker` list, mentor reviews (take notes), and one rehearsal before lunch. If it wasn't: follow the D2-1 contingency the organizers agreed to.
Last hour	Isaac	Merge only what's tested, tag `v0.1.0-day`, and push.
Judging (if on the 29th)	Dave speaks	Appendix F. Isaac takes the technical questions; Patrick the UI and demo questions.

After the PRD timeline: Fri 30 – Sat 31 Oct: Demo showcase

The conference demo is on Oct 31

- **Oct 30:** rest, then one rehearsal in the evening. Charge everything. Reset `old-cln`.
- **Oct 31:** arrive 60 min early and repeat the Day 9 setup rows. Pitch from Appendix F. Afterwards, file the lightning-vulns issue if the team agreed (D6-4), and ask the Btrust grants team about next steps.

Appendix A: Repo status at commit 79ce029

Area	Status	Detail
Workspace (core, connectors, cli)	DONE	32 tests pass; clippy and fmt clean; Rust 2021 edition.
Advisory format, loader, validation	DONE	Rejects unknown fields, missing operators, a fix inside the range, non-https sources, and duplicate IDs.
Version parsing (CLN, LND, rc, suffixes)	DONE	`<, <=, >, >=, =` joined by commas (AND). No OR. That's fine for the advisories chosen here.
Engine: three verdicts, four condition kinds	DONE	version_range, config_enabled, feature_bit, manual.
Report JSON (the dashboard contract)	DONE	See Appendix C. An unreachable node is `cant_tell`, never green.
CLN connector (clnrest + rune, read-only probe)	CHECK	Unit-tested only. Refuses a rune that can call `listfunds`. Live run is D2-2.
CLI: `validate`, `check` (live / `--nodes` snapshot / `--json`)	DONE	Exit codes 0 / 1 error / 2 can't tell / 3 affected.
Polar CLN images 26.06.7 / 26.06.9	CHECK	`demo/polar/Dockerfile.cln`. Build and run in D2-2.
Advisories	PARTLY	4 verified, 2 unverified; no Lightning Labs entries yet (D3-5).
LND connector, commit_hash, release table	OPEN	D3-1, D4-1.
`serve` API, re-check loop	OPEN	D4-2.
Frontend `web/`	OPEN	D2-4, D3-7, D4-3, D5-3, D6-3.
CI	OPEN	D2-5.
Nostr alerts, exposure check	STRETCH	D6-1, D6-2.
Docs	PARTLY	README is good. `docs/googledocs.txt` is empty, and `docs/Firewatch-Project-Brief.pdf` is the old brief (D8-2).

Appendix B: Advisory data and test cases

B1. Existing files (keep, finish per D3-6)

ID	Impl	Range	Status	Test cases already in `tests/advisories.rs`
FW-2026-001	LDK	`< 0.2.6`, manual	verified	0.2.5 → Can't tell · 0.2.6 → Not affected
FW-2026-002	LDK	`< 0.2.6`, manual	verified	0.2.5 → Can't tell · 0.2.6 → Not affected
FW-2026-003	CLN	`< 26.06.8`, exploited	verified	26.06.7 → Affected · 26.06.8rc1 → Affected · 26.06.8 / 26.06.9 → Not affected
FW-2026-004	CLN	`< 26.06`	UNVERIFIED	Resolve or delete (D3-6)
FW-2026-005	LND	`< 0.21.0`, manual	UNVERIFIED	Not in the demo unless verified (D3-6)
FW-2026-006	CLN	`< 26.06.9`	verified	26.06.7 / 26.06.8 → Affected · 26.06.9 → Not affected

B2. New Lightning Labs files (D3-5). Re-copy every range from the advisory page before writing it

ID	Advisory (Lightning Labs, T1)	`affected` / `fixed`	Condition	Test cases to add

FW-2026-007	update_fee breach fee-burn exploit	`< 0.18.3` / `0.18.3-beta`	version only	0.17.5-beta → Affected · 0.18.2-beta → Affected · 0.18.3-beta → Not · 0.18.5-beta → Not
FW-2026-008	Pending commitment excessive HTLC failback	`< 0.18.0` / `0.18.0-beta`	version only	0.17.5-beta → Affected · 0.18.0-beta → Not
FW-2026-009	HTLC first-stage sweep failure due to wallet budget	`>= 0.18.0, < 0.19.0` / `0.19.0-beta`	version only	0.17.5-beta → Not · 0.18.0-beta → Affected · 0.18.5-beta → Affected · 0.19.0-beta → Not
FW-2026-010	Invoice marked settled after HTLC cancelled by interceptor	`>= 0.18.4, < 0.19.0` / `0.19.0-beta`	`manual` (interceptor in use)	0.18.3-beta → Not · 0.18.4-beta → Can't tell · 0.18.5-beta → Can't tell · 0.19.0-beta → Not

Ranges are as summarized from security.lightning.engineering on Oct 21. The source page wins over this table. FW-2026-010's range assumes no 0.18.6 release exists; confirm that. Left out on purpose: the T3 "DNS seed panic" advisory, which needs an OR range (`< 0.20.2 || = 0.21.0`) and two fixed versions. Add it only if Isaac extends `VersionReq` with `||` and `fixed` becomes a list.

Appendix C: JSON contract (what `check --json` and `/api/report` return)

This is the shape in `crates/firewatch-core/src/report.rs` today. It replaces the PRD's sketch: it has `nodes[]` instead of a single `node`, `findings` instead of `results`, `status` instead of `verdict`, `source` instead of `source_url`, and `steps` instead of `remediation`.

```
// web/src/types/report.ts
export type Status = "affected" | "cant_tell" | "not_affected";
export type Severity = "low" | "medium" | "high" | "critical";
export type Action = "upgrade" | "offline" | "disable_feature";
export type Implementation = "cln" | "lnd" | "ldk";

export interface Finding {
  advisory_id: string;          // "FW-2026-003"
  title: string;
  severity: Severity;
  upstream_severity?: string | null;   // planned (D3-2), e.g. "T1"
  exploited_in_wild: boolean;
  verified: boolean;
  fixed: string | null;         // "26.06.8"; null = no fix yet
  source: string;               // official URL
  status: Status;
  reason: string;               // one sentence, always shown
  manual_checks: string[];      // non-empty when status = "cant_tell"
  action: Action;
  steps: string;                // quoted from source; show when "affected"
}

export type BuildCheck =        // planned (D4-1)
  | { status: "match" } | { status: "unknown_release" } | { status: "not_checked" }
  | { status: "mismatch"; expected: string; actual: string };

export interface NodeReport {
  name: string;
  implementation: Implementation;
  version: string | null;       // null when the node couldn't be read
  network: string | null;
  status: Status;               // worst of findings; "cant_tell" if unreadable
  error: string | null;         // set when unreadable
  build_check?: BuildCheck;     // planned (D4-1)
  findings: Finding[];          // already sorted worst first
}

export interface Report {
  checked_at: number;           // unix seconds
  nodes: NodeReport[];
}
```

Isaac: the exact serde shape of `BuildCheck` is yours to choose in D4-1. Whatever it is, update this type in the same PR and tag Patrick.

Appendix D: `firewatch.toml` for the demo (git-ignored)

```
[[nodes]]
kind = "cln"
name = "old-cln"
url = "http://127.0.0.1:<clnrest port from Polar>"
rune_file = "/home/<you>/firewatch-secrets/old-cln.rune"

[[nodes]]
kind = "cln"
name = "current-cln"
url = "http://127.0.0.1:<port>"
rune_file = "/home/<you>/firewatch-secrets/current-cln.rune"

[[nodes]]                                  # needs D3-1
kind = "lnd"
name = "lnd-old"
url = "https://127.0.0.1:<REST port>"
tls_cert = "/home/<you>/.polar/networks/<id>/volumes/lnd/lnd-old/tls.cert"
macaroon = "/home/<you>/.polar/networks/<id>/volumes/lnd/lnd-
old/data/chain/bitcoin/regtest/readonly.macaroon"

[[nodes]]
kind = "lnd"
name = "lnd-new"
# same fields as above
```

Use absolute paths; `~` isn't expanded. Copy the real ports and paths from `docs/demo-env.md` (D2-2).

Appendix E: Demo network and the expected verdict for every node

Node	Version	Node status	Expected findings (verified advisories only)
`old-cln`	v26.06.7	AFFECTED	FW-003 Affected (exploited chip) · FW-006 Affected · both quote the release notes and link the source
`old-cln` after the upgrade	v26.06.9	NOT AFFECTED	FW-003 "includes the fix (fixed in 26.06.8)" · FW-006 "includes the fix (fixed in 26.06.9)"
`current-cln`	v26.06.9	NOT AFFECTED	As above. Optional node, since `old-cln` after the upgrade shows the same.
`lnd-old`	0.18.5-beta	AFFECTED	FW-009 Affected · FW-010 Can't tell (interceptor hand check) · FW-007, FW-008 Not affected · build_check match
`lnd-old` if Polar only has 0.18.0–0.18.2	0.18.x	AFFECTED	FW-007 Affected · FW-009 Affected · FW-010 Not affected · no amber from this node, so the amber comes from `lnd-unofficial`
`lnd-new`	≥ 0.19.0	NOT AFFECTED	All four LND advisories Not affected · build_check match
`lnd-unofficial`	same as lnd-new	CAN'T TELL	build_check mismatch → every finding Can't tell with the build reason (D4-6)

Appendix F: Demo script (≤ 3:00)

Time	On screen	Dave says / team does
0:00–0:15	Slide 1	"On October 2, Core Lightning warned that attackers were targeting unpatched nodes. In August, attackers stole funds from LND nodes behind BTCPay. Would you have known your node was one of them?"

0:15–0:30	Slide 3	"Firewatch reads your node with read-only credentials, checks it against the developers' own advisories, and gives one of three answers."
0:30–1:10	Dashboard	Patrick presses **Re-check now**. "This node runs Core Lightning 26.06.7: red, two advisories, one of them exploited in the wild. That's the developers' own sentence, and here's their link. We never write the fix ourselves." (If D6-2 works, the phone buzzes; Patrick holds it up.)
1:10–1:45	lnd-old / lnd-unofficial cards	"This LND node is red on a Lightning Labs advisory, and amber on another. That one only matters if you use the HTLC interceptor, which no RPC reveals, so we tell you what to check instead of guessing. And this node *says* it's the latest version, but its build doesn't match the official release, so Firewatch refuses to call it safe. We never show green when we're not sure."
1:45–2:30	Terminal, then dashboard	Isaac runs the upgrade (runbook). Patrick presses Re-check now. The card turns green with "includes the fix". "Same node, same data, upgraded, green, and it says why."
2:30–3:00	Slides 5–6	"Read-only by design. Official sources only. Open format, proposed upstream to the community advisory list. Built for the node operators Kaduna's bootcamps are training. Fixes ship in days; Firewatch makes sure you hear in minutes."

Appendix G: Judge Q&A (owner answers)

Question	Owner	Answer
Isn't this just Lightning Labs' advisory page?	Dave	That page lists LND advisories. We read it, add Core Lightning and LDK, check *your* node's version, build and config, and tell you what applies.
How do you know your data is right?	Dave	Every entry comes from the implementation's own source, is quoted word for word, is audited by Ctrl+F, and is reviewed by a second person. Unverified entries don't run by default.
What if Firewatch is hacked?	Isaac	It holds a rune limited to getinfo and listconfigs, and LND's readonly.macaroon. It refuses broader credentials, and it can't spend or change anything.
What if the version string lies?	Isaac	For LND we compare the commit hash with the official release tag. A mismatch is Can't tell. (Show `lnd-unofficial`.)
Why amber instead of a best guess?	Isaac	A wrong green is worse than no answer. Amber says exactly what to check by hand.
Why not just watch GitHub releases or Umbrel?	Dave	They say an update exists. We say whether you're affected, and what the developers said to do.
Does cargo-audit cover LDK?	Isaac	We found no RustSec entries for LDK's `lightning` crate, and cargo-audit can't see running nodes anyway.
Who maintains the advisories?	Dave	A public repo, one file per advisory, with a test per file. We've proposed the format upstream to the community list.
Why does the UI look like this?	Patrick	One glance per node: icon + word + colour. The fix is quoted with its source, and the hand checks are listed only where needed.
Why Kaduna?	Dave	Bootcamps here are producing new node operators who don't follow developer Discords. The warning has to reach their node.

Appendix H: Packing list for Oct 29 and 31

☐ Demo laptop and charger ☐ Second laptop (backup) and charger ☐ Power bank ☐ Extension board
☐ HDMI cable, plus USB-C → HDMI adapter ☐ Phone hotspot with data ☐ USB stick: backup video + deck PDF + repo zip
☐ Docker images pre-pulled; release binary and `web/dist` built ☐ Printed: script, Q&A sheet, runbook ☐ Phone with the Nostr client logged in